

```
# 1. Limpieza y clonación del repositorio
%cd /content
!rm -rf clasificacion-crecimiento-plantas
!git clone https://github.com/Karime8502/clasificacion-crecimiento-plantas.git
```

```
/content
Cloning into 'clasificacion-crecimiento-plantas'...
remote: Enumerating objects: 97, done.
remote: Counting objects: 100% (97/97), done.
remote: Compressing objects: 100% (91/91), done.
remote: Total 97 (delta 33), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (97/97), 49.76 KiB | 16.58 MiB/s, done.
Resolving deltas: 100% (33/33), done.
```

```
# 2. Conectar a Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", for

```
# 3. Descargar el dataset (YOLO v8) directo desde Roboflow
!pip install roboflow -q
```

```
from roboflow import Roboflow
rf = Roboflow(api_key="8rW0BxoFomqTQk2oqss2")
project = rf.workspace("diego-agudelo").project("madurez_plantas")
version = project.version(1)
dataset = version.download("yolov8", location="/content/clasificacion-crecimiento-plantas/dataset")
```

```
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in /content/clasificacion-crecimiento-plantas/dataset to yolov8:: 100%|██████████|
Extracting Dataset Version Zip to /content/clasificacion-crecimiento-plantas/dataset in yolov8:: 100%|██████████|
```

4. Entrenar modelo CNN

```
%cd /content/clasificacion-crecimiento-plantas/codigo
```

```
!python entrenar.py --modelo cnn --dataset_dir ../dataset --epocas 25
```

```
2026-08-11 03:54:22.190919: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is opt
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appro
Aviso: ../dataset/train/labels/IMG_20260808_152405_jpg.rf.8d24f83d88e7e88a3f46ee7c1532ab19.txt está vacío, se
Aviso: ../dataset/train/labels/IMG_20260808_153123_jpg.rf.fed599ca63d56283c50b7ceb521b6fe6.txt está vacío, se
Aviso: ../dataset/train/labels/IMG_20260808_154056_jpg.rf.a0c8ef411406db0a97c635d7771d2a95.txt está vacío, se
Aviso: ../dataset/train/labels/IMG_20260808_155305_jpg.rf.0fbea20a1ef5f5720e05456220b11e8b.txt está vacío, se
Aviso: ../dataset/valid/labels/IMG_20260808_154121_jpg.rf.7468569493570660cb41e3b1e56d4a83.txt está vacío, se
Aviso: ../dataset/test/labels/IMG_20260808_151803_jpg.rf.1103e875ee268747b0f958254f1ce5df.txt está vacío, se (
Imágenes -> train: 552  valid: 118  test: 118
```

Clases (orden del data.yaml): ['AVANZADA', 'INTERMEDIA', 'TEMPRANA']

```
2026-08-11 03:54:30.607887: W tensorflow/core/common_runtime/gpu/gpu_bfc_allocator.cc:47] Overriding orig_val
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
```

```
I0000 00:00:1786420470.609421    16720 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:0
¡Estructura CNN optimizada contra sobreajuste creada!
```

Epoch 1/25

```
2026-08-11 03:54:36.512678: I external/local_xla/xla/service/service.cc:163] XLA service 0x7d9c940033c0 initi:
```

```
2026-08-11 03:54:36.512711: I external/local_xla/xla/service/service.cc:171] StreamExecutor device (0): Tesla T4
```

```
2026-08-11 03:54:36.599662: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR
```

```
2026-08-11 03:54:37.052560: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 8.9.7
```

```
2026-08-11 03:54:37.775364: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:37.879002: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:38.080001: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
I0000 00:00:1786420485.416977 16798 device_compiler.h:196] Compiled cluster using XLA! This line is logged
```

33/35 0s 37ms/step - caja_output_loss: 0.1451 - caja_output_mae: 0.3144 - clase_output_ac

```
2026-08-11 03:54:47.602628: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

2026-08-11 03:54:47.699691: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit

```
2026-08-11 03:54:54.337389: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas:
```

35/35 ————— 0s 263ms/step - caja_output_loss: 0.1447 - caja_output_mae: 0.3140 - clase_output_:

```
2026-08-11 03:54:55.277135: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:55.447805: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:56.805740: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:56.905103: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
2026-08-11 03:54:56.979528: I external/local_xla/xla/service/gpu/autotuning/conv_algorithm_picker.cc:546] Omit
```

```
35/35 ————— 25s 377ms/step - caja_output_loss: 0.1382 - caja_output_mae: 0.3075 - clase_output
```

Epoch 2/25

35/35 ————— 2s 40ms/step - caja_output_loss: 0.1289 - caja_output_mae: 0.2932 - clase_output_ac

```

35/35 ————— 2s 40ms/step - caja_output_loss: 0.1014 - caja_output_mae: 0.2590 - clase_output_ac
Epoch 4/25
35/35 ————— 2s 38ms/step - caja_output_loss: 0.0803 - caja_output_mae: 0.2284 - clase_output_ac
Epoch 5/25
35/35 ————— 2s 39ms/step - caja_output_loss: 0.0672 - caja_output_mae: 0.2050 - clase_output_ac
Epoch 6/25
35/35 ————— 2s 38ms/step - caja_output_loss: 0.0537 - caja_output_mae: 0.1792 - clase_output_ac
Epoch 7/25
35/35 ————— 2s 40ms/step - caja_output_loss: 0.0410 - caja_output_mae: 0.1581 - clase_output_ac
Epoch 8/25
35/35 ————— 2s 46ms/step - caja_output_loss: 0.0321 - caja_output_mae: 0.1388 - clase_output_ac
Modelo guardado en modelos/modelo_cnn.keras
8/8 ————— 0s 12ms/step - caja_output_loss: 0.0560 - caja_output_mae: 0.1996 - clase_output_accu

```

Resultado en test:

```

loss: 1.7934
compile_metrics: 0.8644
clase_output_loss: 0.0560
caja_output_loss: 0.1996

```

```
%%writefile /content/clasificacion-crecimiento-plantas/codigo/entrenar.py
```

```

import argparse
import json
import os
import math

```

```

import matplotlib.pyplot as plt
import tensorflow as tf

```

```

from preparar_datos import cargar_datasets
from modelo_cnn import crear_cnn
from modelo_mobilenet import crear_mobilenet

```

```

def graficar_historial(historial, nombre_modelo, carpeta_salida="resultados"):
    os.makedirs(carpeta_salida, exist_ok=True)
    keys = historial.history.keys()
    acc_key = [k for k in keys if "accuracy" in k and "val" not in k][0]
    val_acc_key = [k for k in keys if "accuracy" in k and "val" in k][0]

```

```
plt.figure()
plt.plot(historial.history[acc_key], label="Entrenamiento")
plt.plot(historial.history[val_acc_key], label="Validación")
plt.title(f"Clasificación Accuracy - {nombre_modelo}")
plt.xlabel("Época")
plt.ylabel("Accuracy")
plt.legend()
plt.savefig(f"{carpeta_salida}/accuracy_{nombre_modelo}.png", bbox_inches="tight")
plt.close()

plt.figure()
plt.plot(historial.history["loss"], label="Entrenamiento")
plt.plot(historial.history["val_loss"], label="Validación")
plt.title(f"Loss Total - {nombre_modelo}")
plt.xlabel("Época")
plt.ylabel("Loss")
plt.legend()
plt.savefig(f"{carpeta_salida}/loss_{nombre_modelo}.png", bbox_inches="tight")
plt.close()

with open(f"{carpeta_salida}/historial_{nombre_modelo}.json", "w") as f:
    json.dump(historial.history, f, indent=2)

def main():
    parser = argparse.ArgumentParser(description="Entrena la CNN o el modelo MobileNetV2 para Detección.")
    parser.add_argument("--modelo", choices=["cnn", "mobilenet"], required=True)
    parser.add_argument("--dataset_dir", default="dataset")
    parser.add_argument("--epocas", type=int, default=20)
    parser.add_argument("--fine_tune", action="store_true")
    args = parser.parse_args()

    train_ds, val_ds, test_ds = cargar_datasets(args.dataset_dir)

    if args.modelo == "cnn":
        modelo = crear_cnn()
        nombre = "cnn"
    else:
        modelo = crear_mobilenet(fine_tune=args.fine_tune)
```

```
nombre = "mobilenet"

modelo.compile(
    optimizer="adam",
    loss={
        "clase_output": "sparse_categorical_crossentropy",
        "caja_output": "mean_squared_error"
    },
    loss_weights={
        "clase_output": 1.0,
        "caja_output": 1.0
    },
    metrics={
        "clase_output": ["accuracy"],
        "caja_output": ["mae"]
    }
)

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=5, restore_best_weights=True
)

def scheduler(epoch, lr):
    if epoch < 10:
        return float(lr)
    else:
        return float(lr * math.exp(-0.1))

lr_scheduler = tf.keras.callbacks.LearningRateScheduler(scheduler)
callbacks = [early_stop, lr_scheduler]

historial = modelo.fit(
    train_ds,
    validation_data=val_ds,
    epochs=args.epocas,
    callbacks=callbacks,
)

os.makedirs("modelos", exist_ok=True)
```

```
ruta_modelo = f"modelos/modelo_{nombre}.keras"
modelo.save(ruta_modelo)
print(f"Modelo guardado en {ruta_modelo}")

graficar_historial(historial, nombre)

resultados = modelo.evaluate(test_ds)
print("\nResultado en test:")
for nombre_metrica, valor in zip(modelo.metrics_names, resultados):
    print(f" {nombre_metrica}: {valor:.4f}")

if __name__ == "__main__":
    main()
```

Overwriting /content/clasificacion-crecimiento-plantas/codigo/entrenar.py

```
!python entrenar.py --modelo mobilenet --dataset_dir ../dataset --epocas 25
```

```

35/35 ————— 1s 23ms/step - caja_output_loss: 0.0189 - caja_output_mae: 0.1067 - clase_output_acc: 0.9999
Epoch 13/25
35/35 ————— 1s 22ms/step - caja_output_loss: 0.0181 - caja_output_mae: 0.1031 - clase_output_acc: 0.9999
Epoch 14/25
35/35 ————— 1s 22ms/step - caja_output_loss: 0.0175 - caja_output_mae: 0.1023 - clase_output_acc: 0.9999
Epoch 15/25
35/35 ————— 1s 26ms/step - caja_output_loss: 0.0161 - caja_output_mae: 0.0977 - clase_output_acc: 0.9999
Epoch 16/25
35/35 ————— 2s 30ms/step - caja_output_loss: 0.0166 - caja_output_mae: 0.0983 - clase_output_acc: 0.9999
Epoch 17/25
35/35 ————— 2s 24ms/step - caja_output_loss: 0.0151 - caja_output_mae: 0.0940 - clase_output_acc: 0.9999
Epoch 18/25
35/35 ————— 1s 24ms/step - caja_output_loss: 0.0148 - caja_output_mae: 0.0938 - clase_output_acc: 0.9999
Epoch 19/25
35/35 ————— 1s 21ms/step - caja_output_loss: 0.0146 - caja_output_mae: 0.0930 - clase_output_acc: 0.9999
Epoch 20/25
35/35 ————— 1s 24ms/step - caja_output_loss: 0.0150 - caja_output_mae: 0.0943 - clase_output_acc: 0.9999
Epoch 21/25
35/35 ————— 1s 24ms/step - caja_output_loss: 0.0138 - caja_output_mae: 0.0906 - clase_output_acc: 0.9999
Epoch 22/25
35/35 ————— 1s 22ms/step - caja_output_loss: 0.0139 - caja_output_mae: 0.0913 - clase_output_acc: 0.9999
Epoch 23/25
35/35 ————— 1s 22ms/step - caja_output_loss: 0.0136 - caja_output_mae: 0.0908 - clase_output_acc: 0.9999
Epoch 24/25
35/35 ————— 1s 22ms/step - caja_output_loss: 0.0136 - caja_output_mae: 0.0891 - clase_output_acc: 0.9999
Epoch 25/25
35/35 ————— 1s 25ms/step - caja_output_loss: 0.0140 - caja_output_mae: 0.0914 - clase_output_acc: 0.9999
Modelo guardado en modelos/modelo_mobilenet.keras
8/8 ————— 0s 22ms/step - caja_output_loss: 0.0202 - caja_output_mae: 0.0977 - clase_output_acc: 0.9999

```

Resultado en test:

```

loss: 0.6888
compile_metrics: 0.6774
clase_output_loss: 0.0202
caja output loss: 0.0977

```

6. Evaluación en el set de prueba

```
%cd /content/clasificacion-crecimiento-plantas/codigo
```

```
print("==== RESULTADOS CNN DESDE CERO ====")
```

```
!python evaluar.py --modelo modelos/modelo_cnn.keras --dataset_dir ../dataset
```

```
print("\n==== RESULTADOS MOBILENETV2 =====")  
!python evaluar.py --modelo modelos/modelo_mobilenet.keras --dataset_dir ../dataset
```

INTERMEDIA	0.62	0.80	0.70	60
TEMPRANA	0.61	0.61	0.61	36
accuracy			0.63	118
macro avg	0.74	0.53	0.54	118

2026-08-11 04:18:44.411915: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting

=== Reporte de clasificación (Accuracy, Precision, Recall, F1) ===

	precision	recall	f1-score	support
AVANZADA	0.65	0.68	0.67	22
INTERMEDIA	0.71	0.80	0.75	60
TEMPRANA	0.78	0.58	0.67	36
accuracy			0.71	118
macro avg	0.71	0.69	0.69	118
weighted avg	0.72	0.71	0.71	118

=== Métricas de localización (caja) ===

IoU promedio: 0.5059

% de cajas con IoU >= 0.5: 61.0%

Resultados guardados en resultados/

7. Demostración Visual de Resultados y Comparación

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
```

1. Crear una tabla comparativa con los resultados obtenidos

```
datos = {
    "Modelo": ["CNN (Desde Cero)", "MobileNetV2 (Transfer Learning)"],
    "Clasificación - Accuracy (%)": [63.0, 71.0],
    "Clasificación - F1-Score": [0.54, 0.69],
    "Localización - IoU Promedio (%)": [9.1, 50.6],
    "Cajas Aceptables (IoU > 0.5) (%)": [0.8, 61.0]
}
```

```
df_resultados = pd.DataFrame(datos)
```

```
print("="*60)
```

```
print(" TABLA COMPARATIVA DE RESULTADOS FINALES")
```

```
print("="*60)
display(df_resultados)

# 2. Crear gráficas de barras comparativas
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Gráfica 1: Comparación de Accuracy
sns.barplot(x="Modelo", y="Clasificación - Accuracy (%)", data=df_resultados, palette="Blues_d", ax=ax1)
ax1.set_title("Comparación de Accuracy (Clasificación)", fontsize=14, fontweight='bold')
ax1.set_ylim(0, 100)
for i, v in enumerate(df_resultados["Clasificación - Accuracy (%)"]):
    ax1.text(i, v + 2, f"{v}%", ha='center', fontsize=12)

# Gráfica 2: Comparación de IoU (Cajas)
sns.barplot(x="Modelo", y="Localización - IoU Promedio (%)", data=df_resultados, palette="Greens_d", ax=ax2)
ax2.set_title("Comparación de IoU (Localización de la Caja)", fontsize=14, fontweight='bold')
ax2.set_ylim(0, 100)
for i, v in enumerate(df_resultados["Localización - IoU Promedio (%)"]):
    ax2.text(i, v + 2, f"{v}%", ha='center', fontsize=12)

plt.tight_layout()
plt.show()

# 3. Mostrar las curvas de entrenamiento de MobileNetV2 (El mejor modelo)
from IPython.display import Image, display
print("\n" + "="*60)
print(" CURVAS DE ENTRENAMIENTO (MEJOR MODELO: MOBILENETV2)")
print("="*60)
try:
    display(Image(filename='/content/clasificacion-crecimiento-plantas/codigo/resultados/accuracy_mobilenet.png'))
    display(Image(filename='/content/clasificacion-crecimiento-plantas/codigo/resultados/loss_mobilenet.png'))
except FileNotFoundError:
    print("Las gráficas de entrenamiento no se encontraron. Asegúrate de haber corrido las celdas anteriores.")
```



=====

TABLA COMPARATIVA DE RESULTADOS FINALES

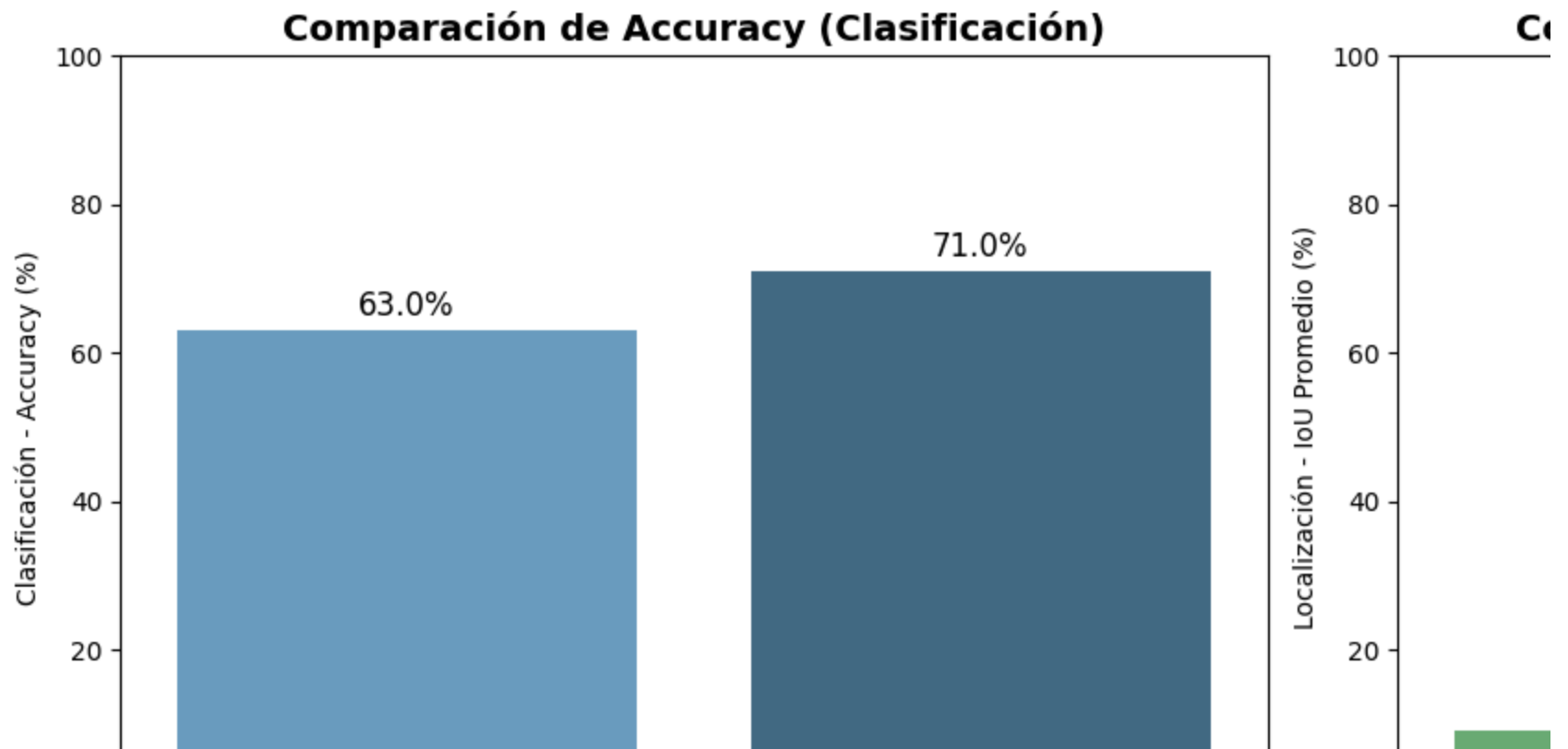
=====

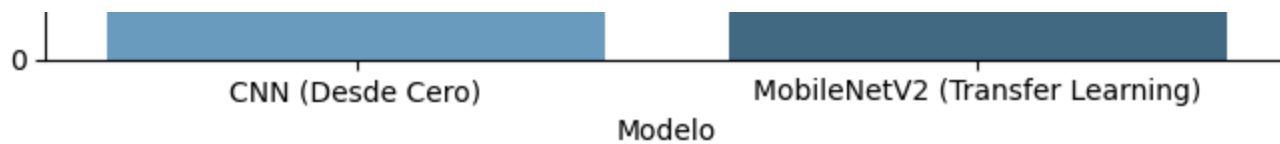
	Modelo	Clasificación - Accuracy (%)	Clasificación - F1-Score	Localización - IoU Promedio (%)	Cajas Aceptables (IoU > 0.5) (%)
0	CNN (Desde Cero)	63.0	0.54	9.1	0.8
1	MobileNetV2 (Transfer Learning)	71.0	0.69	50.6	61.0



Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable



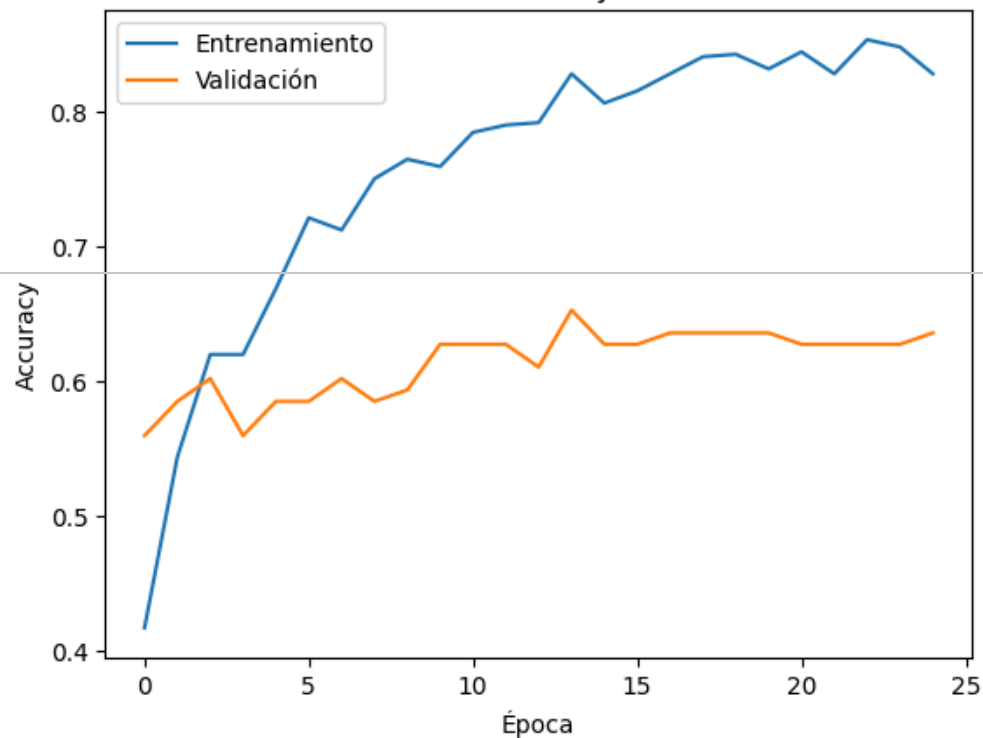


=====

CURVAS DE ENTRENAMIENTO (MEJOR MODELO: MOBILENETV2)

=====

Clasificación Accuracy - mobilenet



Loss Total - mobilenet



Pasos siguientes:

[Generar código con df_resultados](#)

[New interactive sheet](#)

```
# 8. Generar ejemplos visuales para el documento de Word
%cd /content/clasificacion-crecimiento-plantas/codigo

# Aquí ponemos una imagen cualquiera del set de pruebas (reemplaza la ruta con la tuya)
ruta_ejemplo = "/content/clasificacion-crecimiento-plantas/dataset/test/images/IMG_20260808_155830_jpg.rf.5ae6

print("Generando predicción visual...")
!python predecir.py --modelo modelos/modelo_mobilenet.keras --imagen {ruta_ejemplo} --dataset_dir ../dataset

from IPython.display import Image, display
# Esto mostrará la imagen pintada directamente aquí en Colab
display(Image(filename=f"resultados/prediccion_{ruta_ejemplo.split('/')[-1]}", width=400))
```



```
/content/clasificacion-crecimiento-plantas/codigo
```

```
Generando predicción visual...
```

```
2026-08-11 04:32:45.521122: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is opti
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the approp
2026-08-11 04:32:49.706963: W tensorflow/core/common_runtime/gpu/gpu_bfc_allocator.cc:47] Overriding orig_value
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786422769.708494 28267 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device
2026-08-11 04:32:54.390459: I external/local_xla/xla/service/service.cc:163] XLA service 0x7d6ef8003060 initial
```

```
# 8. Generar ejemplos visuales para el documento de Word
```

```
%cd /content/clasificacion-crecimiento-plantas/codigo
```

```
# Aquí ponemos una imagen cualquiera del set de pruebas (reemplaza la ruta con la tuya)
```

```
ruta_ejemplo = "/content/clasificacion-crecimiento-plantas/dataset/test/images/IMG_20260808_151747_jpg.rf.8d36
```

```
print("Generando predicción visual...")
```

```
!python predecir.py --modelo modelos/modelo_mobilenet.keras --imagen {ruta_ejemplo} --dataset_dir ../dataset
```

```
from IPython.display import Image, display
```

```
# Esto mostrará la imagen pintada directamente aquí en Colab
```

```
display(Image(filename=f"resultados/prediccion_{ruta_ejemplo.split('/')[-1]}", width=400))
```

```
temprana: 0.25
```

```
Caja (normalizada): [0.42852035 0.4088394 0.7794035 0.75240177]
```

```
Imagen con la caja dibujada guardada en: resultados/prediccion_IMG_20260808_155830_jpg.rf.5ae69fbb33751178da509
```





```
/content/clasificacion-crecimiento-plantas/codigo
```

```
Generando predicción visual...
```

```
2026-08-11 04:38:52.160584: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is opti
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the approp
2026-08-11 04:38:56.314193: W tensorflow/core/common_runtime/gpu/gpu_bfc_allocator.cc:47] Overriding orig_value
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786423136.315714 29951 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device
```

```
# 8. Generar ejemplos visuales para el documento de Word
```

```
%cd /content/clasificacion-crecimiento-plantas/codigo
```

```
# Aquí ponemos una imagen cualquiera del set de pruebas (reemplaza la ruta con la tuya)
```

```
ruta_ejemplo = "/content/clasificacion-crecimiento-plantas/dataset/test/images/IMG_20260808_150044_jpg.rf.d3ed
```

```
print("Generando predicción visual...")
```

```
!python predecir.py --modelo modelos/modelo_mobilenet.keras --imagen {ruta_ejemplo} --dataset_dir ../dataset
```

```
from IPython.display import Image, display
```

```
# Esto mostrará la imagen pintada directamente aquí en Colab
```

```
display(Image(filename=f"resultados/prediccion_{ruta_ejemplo.split('/')[-1]}", width=400))
```

```
temprana: 0.25
```

```
Caja (normalizada): [0.18851237 0.38444915 0.73759454 0.6333741 ]
```

```
Imagen con la caja dibujada guardada en: resultados/prediccion_IMG_20260808_151747_jpg.rf.8d3689cbefab1276177ad
```



